Data Engineer - Technical Assessment Snore MD

Name: Syed Muhammad Zaid

Email: zaid.syed1116@gmail.com

Part 3: Engineering Reasoning & Communication (Written)

- **Scalability**

  At higher scale, the first bottleneck would be ingestion. In 'base_ingester.py', I'm currently using 'cursor.executemany()', which still executes one 'INSERT' per row. With ~350 appointments and ~200 sleep studies per batch, this is acceptable. But at 10× scale (~3,500 appointments per batch), the per-row round-trip to Snowflake becomes the constraint. The limiting factor wouldn't be Snowflake's compute, but the network and transaction overhead of issuing thousands of individual inserts.

  The second bottleneck would be the X-SMALL warehouse defined in 'setup.sql'. At 10× volume, the dbt mart models would be aggregating over hundreds of thousands of fact rows instead of tens of thousands, which would likely push runtimes from seconds to minutes. The 60-second auto-suspend could also introduce cold-start latency if the warehouse suspends between runs. To address this, I would switch ingestion to a staged bulk load using 'COPY INTO' (writing the batch to Parquet in S3 and loading in parallel), and dynamically scale the warehouse up to MEDIUM at the start of a dbt run before scaling it back down after completion to manage cost.

- **Data Governance**

  I already implemented **data lineage** in Part 2 at the model level using dbt's 'ref()' and 'source()' macros. In my project, each model explicitly defines its upstream dependencies. For example, 'stg_patients' sources from ' RAW.PATIENTS', 'dim_patients' references ' stg_patients', and 'fct_appointments' references 'stg_appointments' along with the relevant dimension models. Because I declare these dependencies directly in code, the full lineage path from RAW through STAGING to ANALYTICS is machine-readable and version-controlled in Git. When I run ' dbt docs generate && dbt docs serve', I can see a lineage graph that clearly shows how each model is built and which upstream sources feed it, without manually tracing SQL.

  In a production setting, I would integrate the dbt artifacts (manifest.json and run_results.json) generated by the CI/CD pipeline into a data catalog such as DataHub. These platforms can automatically ingest dbt lineage and combine it with Snowflake metadata and business glossary definitions, giving stakeholders a centralized view of data flow. This would allow me to quickly answer governance questions such as where a KPI originates and how it is derived.

  I enforce **column-level security** in snowflake_setup/rbac.sql using Snowflake Dynamic Data Masking policies. I defined two masking policies: phi_string_mask for email and phone, and phi_date_mask for date of birth. The string masking policy returns the true value only when the active role is' DE_ROLE', 'ACCOUNTADMIN', or ' SYSADMIN'. For all other roles, including ANALYST_ROLE and CLINIC_ROLE, the value is replaced with '****MASKED****'. The date

masking policy returns NULL for non-DE roles, ensuring that date of birth is not directly exposed; instead, analysts use the derived age column in ' dim_patients'. Because I implemented masking at the Snowflake layer, it is enforced consistently regardless of how the data is queried (dbt, Retool, direct SQL, or a BI tool).

I implemented **audit logging** at two levels. First, the RAW.INGESTION_LOG table is written by base_ingester.py on each pipeline execution. It records the source name, file name, number of rows ingested, run status (success or failed), any error message, and a timestamp. This gives me a clear operational history of ingestion activity and supports troubleshooting and compliance reporting. Second, I rely on Snowflake's ACCOUNT_USAGE.ACCESS_HISTORY view to capture query-level access logs, including which users accessed which tables and columns and when. In a production environment, I would also enable infrastructure-level logging (AWS CloudTrail or Azure Monitor) to capture access to storage and the Snowflake account itself, ensuring full end-to-end audit coverage suitable for healthcare compliance requirements.

For **PHI protection**, I follow a defence-in-depth approach across the medallion architecture. In the RAW (Bronze) layer, PHI fields such as email, phone, and date of birth are stored in plaintext, but I restrict access via RBAC to DE_ROLE and the ingestion service account. In the STAGING (Silver) layer, I retain PHI as needed for joins and deduplication before surrogate keys are generated. In the ANALYTICS (Gold) layer, which analysts and Retool access, I transform email and phone into SHA2(value, 256) hashes within dim_patients, making them non-reversible without the original values. I retain date of birth only for age calculation and mask it to NULL for non-privileged roles using the phi_date_mask policy. This layered approach ensures that sensitive information is controlled and appropriately transformed before reaching business users.

- **Tradeoffs: Why I chose these ingestion and warehouse technologies**

Ingestion: Python scripts over AWS Glue or Azure Data Factory:

For this assessment, I used plain Python ingestion scripts (with the Snowflake connector) because they are the most reproducible and easiest to review. I also implemented watermark and incremental ingestion logic in `base_ingester.py` in a cloud-agnostic way. The same logic can be moved into AWS Glue (Python Shell) or into Azure Data Factory as a Python activity later. The business rules stay the same, and only the execution and orchestration layer changes.

The downside is that local Python scripts are not production orchestration. They rely on a machine being available and provide no built-in scheduling, retries, alerting, or operational guardrails. In production the data would not come from generated files but from live sources: the CRM scheduling API, Azure Blob device exports, and the billing system. In that environment I would keep the same ingestion business logic but run it on managed infrastructure Glue for AWS-side sources, ADF for Azure-side sources, triggered by source system events rather than a manual script run, with retries and monitoring handled by the platform.

Warehouse: Snowflake over Redshift or Azure Synapse

I chose Snowflake because it fits the cross-cloud scenario directly. It runs natively on both AWS and Azure, so I can analyze data in one platform without forcing everything into a single cloud provider. If I used Redshift, I would need to move Azure data into AWS first. If I used Synapse, I would need to move AWS data into Azure. Either approach adds cross-cloud transfer costs, additional latency, and extra operational complexity, and it is not ideal when the data can include PHI. The main tradeoff is the cost model. Snowflake's per-credit pricing can be more expensive than reserved capacity on Redshift for very high, sustained workloads. But for this workload pattern, where the warehouse is idle much of the day, Snowflake is cost-efficient because compute can auto-suspend quickly. I set auto-suspend to 60 seconds in 'setup.sql', so compute is billed only while queries and transformations are running.

- **Reliability:**

I detect **partial load** using the INGESTION_LOG table, because it records every source execution with a status and row count, so it's easy to see which sources completed and which did not for a given run. Recovery is handled automatically through the per-source watermark logic in base_ingester.py. On the next run, get_max_watermark() reads MAX(created_at) from each RAW table independently, so sources that already committed their batch are effectively skipped while the failed source reloads only the rows it missed. In practice, the recovery action is simply re-running the pipeline, with no manual deduplication needed. In a production setup, I would make each load atomic by wrapping the write in an explicit Snowflake transaction so a batch either fully commits or rolls back, removing any ambiguity about what the watermark reflects after a crash.

Corrupt files are caught at extraction time. Each ingester's extract() step uses pandas readers like read_csv() and read_json(), which throw exceptions when the input is malformed, for example truncated records, encoding problems, or unexpected columns. The run() method in base_ingester.py wraps the extract–filter–load flow in a try/except, and on any exception it logs the full stack trace to logs/ingestion.log and also writes a failed status into INGESTION_LOG so the failure is visible in Snowflake. Importantly, the pipeline is designed to fail per source, not globally, so a corrupt appointments.json does not prevent other sources like sleep studies or billing reports from loading. Recovery is to fix or replace the corrupted file and re-run the pipeline; the watermark prevents reloading rows that were already successfully committed in a previous run. In production, I would add row-level validation before loading and route invalid rows into quarantine tables (with the failure reason) so clean rows can still load while bad records are surfaced for review.

API schema changes can be silent, meaning the pipeline might still run but produce missing or incorrect downstream fields. My primary detection mechanism is dbt, where the staging models act as a schema contract. If the API renames a field (for example appointment_type to appt_type), the next dbt run fails with a clear column-not-found error in the relevant staging model, preventing the issue from propagating into fact tables or dashboards. On top of that, the GitHub Actions workflow runs dbt run and dbt test on pushes to main, which ensures schema-breaking changes get caught early through CI checks. Recovery is to update the affected staging model to map the new field name and push the fix so CI verifies it before

the next scheduled run. In production, I would also add a proactive schema fingerprint check during ingestion by hashing the set of field names returned by the API (or file headers), comparing it against the last known fingerprint stored in a Snowflake config table, and alerting immediately if it changes, so the team gets notified before dbt even fails.